

Guide family map

Where this guide sits in the full Blueprint Authoring Atlas family.

ROOT & CROSS-CUTTING

Authoring Atlas

Blueprint Bundle

Metamodel Vocabulary

Migrations

DESIGN

Architecture

Concepts

Domain

Dynamics

Infrastructure

Interactions

Models

Quality

Rules

Story

GOVERNANCE

Capability

Decisions

Motivation

Organization

Roadmap

Test Cases

Value Stream

Capture → focus → transformation

What to gather first, what this layer focuses on, and what a modeler or AI agent can produce from it.

Capture inputs

- Workshop notes
- Guide selection
- Cross-layer consistency

Authoring Atlas

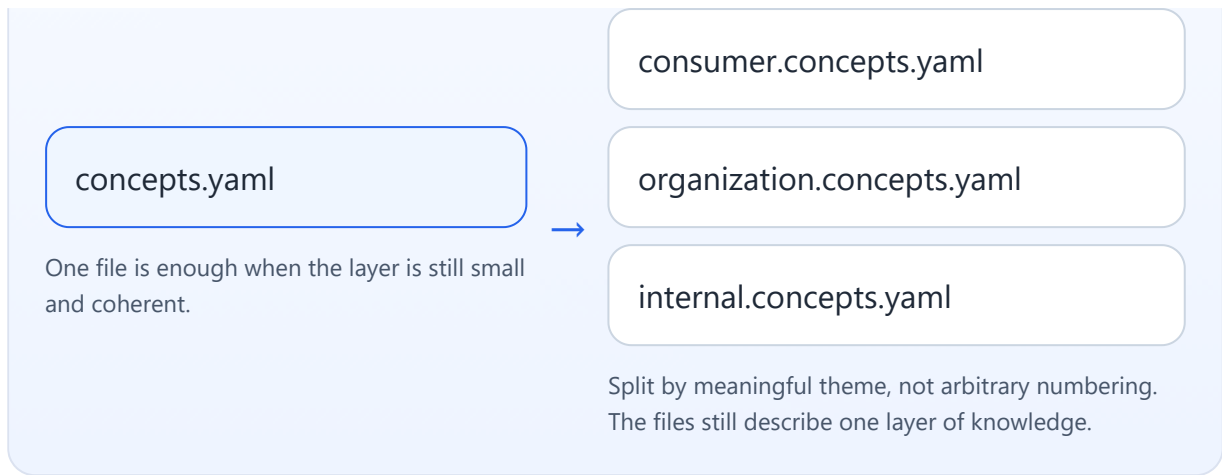
This guide helps non-technical people express this layer clearly.

Transformation outputs

- Transformation-ready text
- Shared guide map
- Review loop

One layer can span multiple files

Use thematic file splits when the knowledge is large, semantically clustered, or owned by different teams.



Blueprint Authoring Atlas

This guide family is for **non-technical contributors** who need to describe a business, process, screen flow, concept, risk, or decision clearly enough that a technical modeler or AI agent can turn that description into Blueprint YAML.

This is **not** a schema reference. It explains **what knowledge to capture, how to express it, and how to hand it off**.

What you are trying to do

Blueprint authoring starts from plain language:

- name the important **things** in the domain
- describe the **relationships** between them
- explain the **flows** and **working steps**
- capture the **screens/interactions** people use
- record **rules, goals, risks, evidence, and decisions**

If that knowledge is clear, a technical person or AI agent can transform it into YAML. If that knowledge is vague, mixed, or contradictory, the YAML will also be vague, mixed, or contradictory.

How to use this guide family

1. Start with the layer that best matches the knowledge you have.
2. Write plain-language notes using the prompts in that guide.
3. Keep one kind of knowledge in one layer as much as possible.
4. Hand the text to a technical modeler or AI agent for YAML transformation.
5. Review the result together and correct missing or wrong meaning.

What makes good input

Good authoring input is:

- **concrete** — it names real actors, things, actions, and outcomes
- **layer-aware** — it does not mix concepts, screens, stories, and risks into one blob
- **connected** — it explains relationships, not only isolated items
- **outcome-focused** — it explains what changes, not only what exists
- **reviewable** — another person can read it and point out gaps or contradictions

The layers and root guides at a glance

Use these guides depending on what you are trying to capture. The family has:

- **root/cross-cutting guides** — how the whole Blueprint bundle hangs together
- **design guides** — what the system is and how it works
- **governance guides** — why it exists, who owns it, how it changes, and how it is proven

Root and cross-cutting guides

Guide	Use it when you need to describe	Status
Blueprint Bundle	the overall system model, its scope, layout, slices, shared files, and bundle-level principles	available
Metamodel Vocabulary	cross-layer vocabulary, typed IDs, shared meanings, and reference consistency	available
Migrations	model change, upgrade intent, ordered changes, rollback, and impact over time	available

Design guides

Guide	Use it when you need to describe	Status
Architecture	system boundaries, parties, contexts, services, and dependencies	available
Concepts	business terms, entities, identities, relationships, and state meaning	available

Domain	operations, events, queries, documents, errors, and key questions	available
Dynamics	runtime ordering, timing, parallel work, and race conditions	available
Infrastructure	environments, resources, topology, and operational ownership	available
Interactions	screens, actions, responses, navigation, and user-system exchanges	available
Models	data shapes crossing boundaries or shown to users	available
Quality	measures, KPIs, SLOs, security, compliance, resilience, and observability	available
Rules	obligations, prohibitions, derivations, validation, and transitions	available
Story	journeys, processes, activities, steps, and outcomes	available

Governance guides

Guide	Use it when you need to describe	Status
Capability	what the business can do, independent of teams or processes	available
Decisions	explicit choices, rationale, status, and impact	available
Motivation	goals, non-goals, risks, assumptions, trade-offs, and open questions	available
Organization	parties, departments, teams, and ownership	available
Roadmap	milestones, deliverables, dependencies, and success criteria	available
Test Cases	scenarios proving expected, edge, error, and fitness behavior	available
Value Stream	end-to-end value stages across capabilities and actors	available

How plain-language capture becomes YAML

There are two roles in the handoff:

- **non-technical contributor**
 - explains the real-world meaning
 - provides examples, context, and relationships
 - confirms whether the transformed output is faithful
- **technical modeler or AI agent**
 - maps the capture into the right Blueprint layer
 - structures the content into YAML
 - resolves references, IDs, and traceability
 - asks follow-up questions where the source text is incomplete

The goal is **faithful transformation**, not inventing meaning that was never captured.

What `blueprint.schema` means for authors

You do **not** need to study `blueprint.schema` directly.

What matters for you is this:

- it describes the overall contract of a Blueprint bundle
- it is why the final model needs enough structure and completeness
- it is why mixed-up or contradictory input creates problems later

In simple terms: it is the reason your input needs to be clear and well separated by layer.

If you need to describe the **whole Blueprint bundle** in plain language, use the dedicated [Blueprint Bundle guide](#).

What `metamodel.schema` means for authors

You do **not** need to read `metamodel.schema` line by line.

What matters for you is this:

- it is the shape behind the Blueprint language
- it is why consistent naming matters
- it is why the same concept should not be described three different ways in three different guides

In simple terms: it is the reason vocabulary and relationships must stay consistent.

If you need to align people on shared **cross-layer vocabulary and typed references**, use the dedicated [Metamodel Vocabulary guide](#).

What migrations mean for authors

Blueprint evolves over time. Migrations exist because the language changes.

For authors, this means:

- older notes may need updating before reuse
- examples from earlier versions may not map perfectly to the latest guide set
- keeping capture clear and plain-language makes migration easier

In simple terms: if the language evolves, clear source thinking survives better than tool-specific wording.

If you need to describe **how the model changes over time**, use the dedicated [Migrations guide](#).

Cross-layer consistency checklist

Before handing off your notes, check:

- are the same **names** used consistently?
- are the same **concepts** described the same way in every guide?
- do **stories**, **interactions**, and **concepts** refer to one another clearly?
- are **actors** named consistently?
- are **outcomes**, **rules**, and **risks** stated without contradiction?

Recommended capture workflow

1. Start with rough notes from interviews, workshops, or observation.
2. Move the notes into the right layer guide.
3. Fill missing actors, relationships, steps, and outcomes.
4. Hand the result to a technical modeler or AI agent.
5. Review the transformed YAML for fidelity.
6. Correct meaning first, formatting second.

When one layer needs multiple files

Even non-technical contributors should know that one Blueprint layer does **not** always mean one physical file.

Sometimes the same schema type is split across multiple files because the knowledge is too large, has natural semantic clusters, or belongs to different ownership groups.

Good reasons to split one layer into multiple files

- the content grows beyond a manageable size
- there are clear semantic clusters
- different teams own different subsets
- you want the file layout to reveal the domain structure more clearly

Examples:

- `consumer.concepts.yaml` and `organization.concepts.yaml`
- `checkout.domain.yaml` and `fulfillment.domain.yaml`
- `pricing.rules.yaml` and `compliance.rules.yaml`
- `happy-path.story.yaml` and `error-handling.story.yaml`

What this means for non-technical authors

You should still think in terms of **one layer of knowledge**, but you may choose to organize your capture into **multiple thematic documents** inside that layer.

For example:

- one Concepts capture for customer-facing terms and another for internal/back-office terms
- one Story capture for the main journey and another for exception handling
- one Rules capture for policy rules and another for compliance rules

Good practice

- split by **meaningful theme**, not arbitrary numbering
- keep the same kind of knowledge together
- use names that explain the cluster in business language
- avoid splitting too early if one clear document is still enough
- keep cross-file vocabulary consistent so the transformed YAML still reads like one model

Important scoping rule

The same schema type can appear:

- at the **root level** for shared/system-wide concerns, and
- inside **slices** for domain-specific concerns

These are related, but they are **not the same scope**.

In plain terms:

- root files describe the forest
- slice files describe the trees

Naming pattern to know

When the technical model is split into multiple files of the same type, the common pattern is:

- `{semantic-prefix}.{schema-type}.yaml`

Examples:

- `consumer.concepts.yaml`
- `checkout.story.yaml`
- `security.decisions.yaml`

You do not need to produce final filenames yourself, but it helps to organize your source notes in the same spirit: by clear semantic cluster.

PDF exports

Generated PDFs for workshop/offline sharing live in [./pdf/](#):

- [Blueprint Authoring Atlas PDF](#)
- one PDF per layer guide, generated from the Markdown sources in this folder

The PDF set also includes root/cross-cutting guides for Blueprint Bundle, Metamodel Vocabulary, and Migrations.

Common mistakes

- mixing screens, concepts, stories, and risks into one document
- describing UI layout but not user intent
- naming activities without outcomes
- naming concepts without relationships

- using examples as if they were the whole rule
- writing implementation details before business meaning is clear

Start here

Use these first:

- [Story authoring guide](#)
- [Interactions authoring guide](#)
- [Concepts authoring guide](#)

Then continue with the rest of the design, governance, and root guides listed above.

